作业 3.2 演讲稿：priority-* 是怎么通过的

配合 `10244602411-刘小凡-lab3-课堂报告第一周.pptx`（12 页）。限时 5 分钟讲解 + 1 分钟提问。

这版稿子的结构：每页先把 PPT 上的基本内容讲清楚，再加一段 PPT 上没有的补充（为什么这样设计、换种写法会怎样、测试为什么抓不到某个 bug），让老师听出你不只是在念 PPT。

标记

- 普通段落：PPT 上的基本内容。
- ▲ **开头的段落**：PPT 上没有的补充，是加分的重点，尽量别跳。
- 〔六角括号〕：更多补充，时间够才讲，超时就跳过，跳过不影响前后衔接。
- 【方括号】：动作提示，不用念出来。**加粗**的词念的时候重读一下。

不讲〔〕的话，正文约 1500 字，语速要偏快一点才能控制在 5 分钟左右；〔〕全讲要多 1 分多钟，所以只挑一两个。练的时候对着下表计时，讲完某一页时如果比表里晚了 15 秒以上，后面的〔〕就都跳过。PPT 备注里还是旧版讲稿，以这份为准。

页	PPT 上的基本内容	▲ 补充	讲完这页时
1	H、M、L，L 被借成 33	本质是调度器和锁怎么配合	0:20
2	AI、12 小时、环境、参考	—	0:35
3	19 个失败，分三类	19 是怎么算出来的	0:58
4	有序队列、三处抢占、等号的坑	其实有五处抢占	1:31
5	panic、backtrace、intr_yield_on_return	真正的切换发生在什么时候	1:57
6	优先级反转、火星探路者、捐赠	有界和无界的区别	2:23
7	按锁记账、公式、两种方案	两种方案差在哪	2:49
8	捐出去、收回来、只改 base	lock_release 顺序反了会怎样（实测）	3:19
9	第一版挂了 4 个，两个错	为什么 donate-one 第一版就能过	3:48
10	donate-sema 的排查和修复	为什么不在捐赠时顺便调整等待队列	4:20
11	感悟	—	4:33
12	结果、三条收获	退化实验；我知道的不足	5:00

第 1 页　开场

大家好，我讲的是 priority 开头的 12 个测试是怎么通过的。

【指右边色块】低优先级的 L 本来是 31，因为拿着高优先级 H 要的锁，被**借**成了 33，这就是今天的重点：优先级捐赠。

▲ priority 这部分本质上是在解决**调度器和锁怎么配合**：只看调度，谁高谁上 CPU；有了锁，高的也可能被低的挡住。

第 2 页　我的 lab 总览

先交代一下：【指 1】AI 我基本只用智能补全，卡住时才用 Claude Code；【指 2】一共大概 12 个小时；【指 3】在 Ubuntu 服务器上用 QEMU 跑；【指 4】也参考了不少别人的代码，比如 B 站的讲解。所以后面我多讲每处改动背后的原因。

第 3 页　从 19 个失败开始

lab2 之后还挂着 **19 个**，priority 占 12 个。我先读了每个测试的源码和期望输出，按“它在问什么”分成三类：调度与抢占、按优先级唤醒、优先级捐赠，一层一层地做。

▲ 19 是能算出来的：alarm 的 6 个 lab2 已经过了；mlfqs-fair-2 和 fair-20 里所有线程 nice 都是 0，普通轮转本来就公平，不写 MLFQS 也能过。27 减 8，正好 19。

〔顺序也不是随便排的：捐赠的效果要靠抢占才能体现出来，所以必须先把调度做对。〕

第 4 页　第一层：排好序，随时抢占

第一层是调度：ready_list 按优先级降序插入，队首就是下一个要跑的；在创建线程、改优先级、sema_up 之后检查，队首比我高就让出 CPU。

第一个坑是**一个等号**。【指队列图】比较函数写成大于等于，yield 的线程就插到同优先级的最前面，又选中自己，【指输出】priority-fifo 输出 15 15 15，同一个线程连跑 16 次。改成严格大于就好了。

▲ 抢占其实不止三处：lock_release 通过 sema_up 间接检查，时钟中断叫醒线程时也要检查。原则是：**凡是可能冒出更高优先级就绪线程的地方，都要检查。**

〔为什么不干脆在 thread_unblock 里抢占？因为 sema_up 是先 unblock、再把 value 加一，要是在 unblock 里就切走，被叫醒的线程一看 value 还是 0，又睡回去了。〕

第 5 页　困难：kernel panic

做抢占时我还把内核搞崩过：在 thread_wake 里直接调 thread_yield，alarm-priority 一跑就 panic，【指红框】断言 `!intr_context()` 失败。backtrace 只有一行，加上 -fno-omit-frame-pointer 保留帧指针，【指调用链】才看到是时钟中断调进来的。thread_wake 在**中断里**执行，不能切线程，要改用 intr_yield_on_return。

▲ 它其实只是**设一个标志**：中断处理程序收尾、清掉“正在处理中断”的标记以后，才检查这个标志去 yield。所以真正的切换，发生在中断刚结束的那一刻。

〔调用链最底下是 idle，因为那时所有线程都在睡，时钟中断打断的是 idle 线程。〕

第 6 页　为什么需要捐赠

第三层是捐赠。【指上图】L 拿着锁，H 来要锁只能阻塞，M 又一直抢占 L，L 放不了锁，H 就被 M 饿着。这就是**优先级反转**，火星探路者号反复重启就是它。【指下图】有了捐赠，H 把 33 借给 L，L 尽快放锁，H 马上就能跑。

▲ 要分清：H 等 L 的临界区是正常的，是**有界**的；M 插进来以后，H 要等多久取决于 M，才变成**无界**。捐赠消除的是后面这段。

〔为什么只有锁捐赠、信号量不捐？锁有持有者，知道该借给谁；信号量谁都能 up，不知道借给谁。〕

第 7 页　我的设计：按锁记账

我的设计是**按锁记账**：线程记它持有的锁，每把锁记“等它的线程里最高的优先级”，实际优先级等于 base 和这些值取最大，【指图】这里 max(31, 32, 33) 等于 33。任何变化都只调 thread_update_priority 重算。课件的捐赠者列表方案，我 11 月 1 号也写了一版，同样全过。

▲ 两种结果一样，差在“还”的时候：捐赠者列表放锁时，要遍历所有捐赠者，挑出等这把锁的删掉；按锁记账，只删一把锁。

〔结构体里 waiting_lock 是一个指针，locks 却是链表：线程一等锁就阻塞了，不可能同时等两把，但可以同时拿好几把。〕

第 8 页　核心代码

【指左边】lock_acquire：锁被占着，先记下我在等这把锁，然后沿着“锁、持有者、持有者在等的锁”一路往下捐，最多 8 层；拿到锁后，重算这把锁上剩下等待者的最高值。【指右边】lock_release：删掉这把锁，再重算。thread_set_priority 只改 base，donate-lower 测的就是这个。

▲ lock_release 的顺序不能反：**必须先重算，再 sema_up**。sema_up 最后会拿我的优先级和被叫醒的线程比，我要是还顶着捐赠来的高优先级，就不会让出。我试过反过来，相关的 9 个测试挂了 7 个，donate-one 直接超时。

〔循环遇到已经比我高的就停，为什么不会漏捐？因为锁上的值能那么高，说明之前已经有人沿这条链捐过，后面的人早被抬上去了。〕

第 9 页　第一版挂了 4 个

捐赠第一版我想得太简单：放锁就恢复 base，只捐给直接持有者，结果挂了 4 个。【指左边】multiple 里 main 拿着 a、b，放掉 b 后 a 上的 32 应该还在，我却回到了 31；【指右边】nest 里 H 等 M、M 等 L，我只捐了一层，L 拿不到 33。改成按锁删除再重算、一路捐到底，就都过了。

▲ 那 donate-one 为什么第一版就能过？因为它只有一把锁、一层捐赠，这两个错误的假设**碰巧都对**。测试过了，只说明它覆盖的情况对了。

〔multiple2 里还有个不拿锁的 c（32），它检查的是放掉 a 以后 main 是不是还保持 36；要是 main 掉回 31，c 就会抢在 main 放 b 之前跑完。〕

第 10 页　最后一个红灯

最后是 donate-sema，我卡得最久。diff 看不出原因，我在 sema_up 里加 printf 打出等待队列，【指输出】看到 low 已经被捐到 36，却还排在 34 的 med 后面：插入时排好的顺序，被**等待期间的捐赠**打乱了。改成 sema_up 时用 list_min 重新找最高的，条件变量同理。

▲ 为什么不在捐赠时顺便挪位置？因为捐赠时**不知道持有者挂在哪条链表上**：同一个 elem，可能在 ready_list、睡眠队列，或者某个信号量的等待队列里。ready_list 是全局的，我挪了；等待队列只能在取出时重新找。

〔printf 调试时还要防递归：printf 要拿控制台的锁，放锁又会调 sema_up。〕

第 11 页　感悟

感悟我挑两点说：调试比写代码花的时间多，学会用 printf 和 backtrace 让内核“开口说话”最有用；内核里没有“差不多”，一个等号、一行 yield，就能让测试挂掉甚至内核崩溃。

〔第三点里的管程：Pintos 的条件变量是 Mesa 风格，signal 只是把等待者放进就绪队列，它醒来还要重新拿锁、重新检查条件。〕

第 12 页　结果与收获

最后，priority 12 个全过，make check 27 个也都过了。

▲ 因为参考过别人的代码，全绿后我把 7 处改动**挨个改回去**，每一处都会挂测试，说明都是必要的。我也清楚它的不足：嵌套超过 8 层不再捐，插入是 O(n)，而且只适用于单核，多核就得换自

旋锁。

收获就是这三条：先读期望输出再写代码；有序插入是有前提的；状态用公式算，不要零碎加减。谢谢大家，欢迎老师提问。

提问环节准备（1 分钟）

老师的问题大多从 PPT 上来，所以按页整理了最可能被问的 20 道。回答的套路：**一句结论 + 一句理由**，能举测试就举一个测试。★ 是最可能被问到的，上台前至少把 ★ 的自己说一遍。

第 2 页（我的 lab 总览）

1. ★ **AI 具体用在哪？参考了别人的代码，你自己理解了吗？** 照实说：哪些是补全写的、哪一步卡住用了 Agent。然后落到“我能讲清每一行”：第 12 页的退化实验就是证据，每处改动改回去会挂哪个测试、为什么挂，我都能说出来。比如 sema_up 只取队首，priority-sema 照样过，要靠 donate-sema 才抓得住。

第 3 页（从 19 个失败开始）

2. **为什么是 19 个？哪 8 个本来就能过？** lab2 已经过了 alarm 的 6 个（alarm-priority 也过了）。mlfqs-fair-2 和 fair-20 不实现 MLFQS 也能过：所有线程 nice 都是 0，普通轮转本来就公平。27 减 8 就是 19。

第 4 页（有序 ready_list + 抢占）

3. ★ **为什么 `>=` 会让同一个线程连跑 16 次？** list_insert_ordered 会把新元素插到第一个“比较为真”的元素前面。用 `>=`，同优先级也算真，yield 的线程就插到同优先级线程的最前面，下一次调度又选中它自己。用 `>`，它排到同优先级的最后，才是轮转。

4. **抢占真的只有三个地方吗？thread_preempt 里为什么要判断 intr_context()？** PPT 上是直接调用 thread_preempt 的三处，另外 lock_release 通过 sema_up 间接检查，时钟中断叫醒线程时也检查。sema_up 可能在中断里被调用，中断里不能 thread_yield，只能用 intr_yield_on_return。

第 5 页（kernel panic）

5. ★ **什么是中断上下文？为什么在里面不能切换线程？** CPU 正在处理外部中断（比如时钟中断）时就是中断上下文。中断处理程序借用被打断线程的栈，不属于任何线程，在这里切走，中断就没法正常返回，被打断线程的现场也会乱。所以 thread_yield 一开头就断言 `!intr_context()`。

6. **intr_yield_on_return 什么时候真正让出？** 它只是设一个标志。中断处理程序收尾时，先把“正在处理中断”的标记清掉、通知中断控制器，再检查这个标志去调 thread_yield，那时已经不在中断上下文了。

第 6 页（优先级反转）

7. ★ **什么是优先级反转？捐赠以后 H 还需要等吗？** 高优先级线程被低优先级线程间接挡住。H 等 L 的临界区是正常的、有界的；M 一直抢占 L，H 的等待就变成无界的。捐赠以后 H 还要等 L 跑完临界区，但 M 插不进来了。

8. **为什么只有锁捐赠，信号量不捐赠？** 锁有持有者，知道是谁挡住了我；信号量没有持有者，谁都可以 up，不知道该捐给谁。Pintos 文档也只要求锁的捐赠。

第 7 页（我的设计）

9. ★ **你说课件的捐赠者列表方案也写了，两种有什么区别？** 算出来的优先级是一样的，区别在“还”的时候：捐赠者列表放锁时要遍历所有捐赠者，挑出在等这把锁的删掉，再在剩下的里取最大；按锁记账只要删掉这一把锁、重算一遍。（如果追问那一版的细节，按自己写的代码讲。）

10. ★ **为什么要 base_priority 和 priority 两个字段？** base 是线程自己设的，priority 是加上捐赠后的实际值。只有一个字段的话，捐赠结束后不知道该回到多少；捐赠期间改优先级也会把捐赠覆盖掉，donate-lower 就是测这个。

第 8 页（核心代码）

11. ★ **循环的四个停止条件是什么？提前停会不会漏捐？** 链走到头、锁没人持有、超过 8 层、锁上的值已经不比我低。最后一个不会漏：锁上的值不比我低，说明之前有优先级不低于我的线程捐过这把锁，那次已经沿链传下去了，后面的人都不比我低。

12. **为什么整个 lock_acquire 要关中断？为什么不用锁来保护？** 捐赠循环要连着读写好几个线程和锁的状态，中途被切走的话，别人可能放锁、改优先级，链就断了。锁本身就是我们在实现的东西，不能用锁来保护锁。单核上关中断最简单。

13. ★ **lock_release 里为什么先重算优先级、再 sema_up？** sema_up 最后会拿“我当前的优先级”和被叫醒的线程比。先 sema_up 的话，我还是被捐赠后的高优先级，不会让出，之后也没有地方再检查，被叫醒的线程只能等时间片用完。我试过反过来：相关的 9 个测试挂了 7 个，donate-one 直接超时。

14. **拿到锁以后为什么要重算这把锁的 max_priority？** 锁上可能还有别人在等，它们原来捐给上一个持有者，现在要改为捐给我；而且旧值可能是我自己等锁时捐上去的，我已经不在等了。

15. **嵌套为什么限 8 层？两个线程死锁了会怎样？** 防止锁链太长或者形成环时一直循环，Pintos 文档说可以设一个合理的上限，比如 8 层；测试里最深的 donate-chain 是 7 层。捐赠解决不

了死锁，两个线程还是永远阻塞，但捐赠循环不会卡死：转一圈后锁上的值已经不比我低，就停了。

第 9 页（第一版挂了 4 个）

16. **donate-nest 的过程讲一下。** main（31）拿着 a；medium（32）拿着 b 去等 a，main 变 32；high（33）去等 b，medium 变 33，再沿着 medium 等的 a 捐给 main，main 也变 33。我第一版只捐一层，main 停在 32。

第 10 页（donate-sema）

17. ★ **donate-sema 的完整过程？** L（32）先拿锁，再去等信号量；M（34）也等信号量，排在 L 前面；H（36）来拿 L 的锁，把 L 捐到 36。main sema_up 时应该叫醒 L：L 放锁后 H 拿到锁跑完，然后 M，最后 L。我第一版取队首，叫醒的是 M。

18. **为什么 priority-sema 测不出这个 bug？** priority-sema 没有捐赠，线程在等待期间优先级不变，插入时的顺序一直是对的。

第 12 页（结果与收获）

19. ★ **退化实验具体怎么做的？** 每次只把一处改回去，重新编译，跑相关的几个测试，看哪个挂。priority 的 7 处是：比较函数用 `>=`、创建线程后不抢占、sema_up 取队首、cond_signal 按先来后到、放锁直接恢复 base、只捐一层、set_priority 直接覆盖 priority。每一处都会挂至少一个测试。

延伸

20. **严格优先级调度会饿死低优先级线程吗？** 会，只要一直有高优先级线程就绪，低优先级的就轮不到。priority 部分不要求解决；MLFQS 用 recent_cpu 让占 CPU 多的线程优先级下降，相当于老化。

真的答不上来：不要硬编。先说清楚自己确定的部分，再说推理方向，最后坦白“这种情况我的代码没有处理，测试也没覆盖”。

更多问题见 `Priority通关笔记.html` 的“老师难题区”。